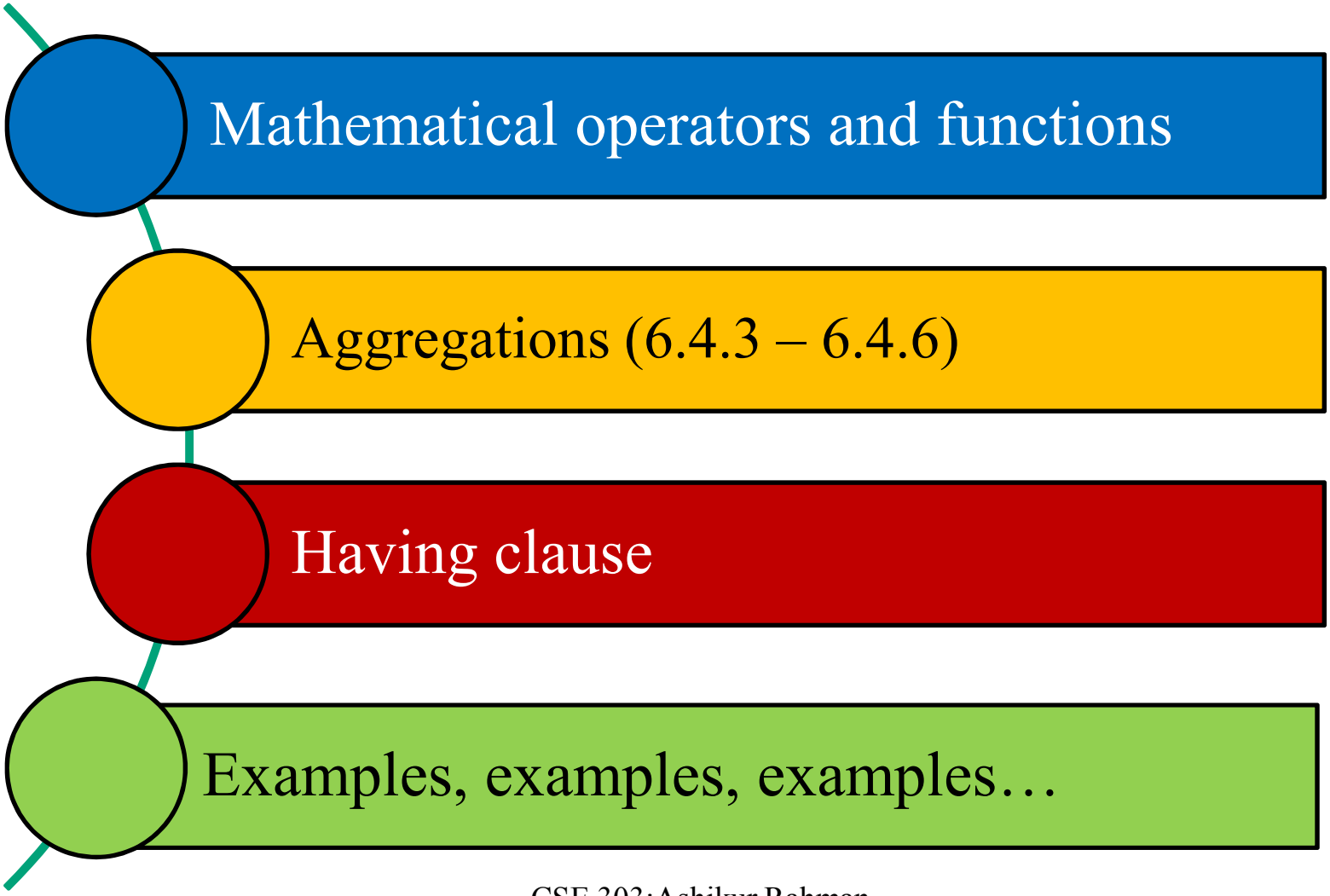
Database Systems

Lecture 05: SQL

2023

Aggregation and Grouping

Outline


Mathematical operators and functions
Aggregations (6.4.3 – 6.4.6)
Having clause
Examples, examples, examples…

Formulas in SQL

So far, only the exact values have been selected from database tables.

It is possible to use formulas where simple literals or column references can appear (`SELECT`, `WHERE`, `SET`, `VALUES` used in `INSERT`, `HAVING`):

```
SELECT (formula on column name) FROM …
```

Formula can be created using an *infix* operator:

```
Function            Definition
--------------------------------
value1 + value2     Addition
value1 - value2     Subtraction
value1 * value2     Multiplication
value1 / value2     Division
```

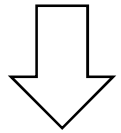

3

Find day-wise sale of each products

Purchase

Product	Day	Price	Quantity
Bagel	10/21/2005	1	20
Banana	10/03/2005	0.5	10
Banana	10/10/2005	1	10
Bagel	10/25/2005	1.50	20

```
SELECT  product,  day, price*quantity AS Sales
FROM    Purchase
```



Product	Day	Sales
Bagel	10/21/2005	20
Banana	10/03/2005	5
Banana	10/10/2005	10
Bagel	10/25/2005	30

NULLS in SQL

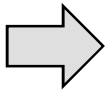
- Whenever we don't have a value, we can put a NULL
- Can mean many things:
  - Value does not exists
  - Value exists but is unknown
  - Value not applicable
  - Etc.
- The schema specifies for each attribute whether it can be null (*nullable* attribute) or not
- How does SQL query cope with tables that have NULLs?

Null Values

Any NULL in a formula will cause a NULL result, not a value of 0.

Marks

student	mark
Paul Smith	43
Rachel Sewell	57
Helen Treacy	32
Pinder Surree	NULL



student	mark	mark/3
Paul Smith	43	14.3333333
Rachel Sewell	57	19
Helen Treacy	72	24
Pinder Surree		

```
SELECT  student, mark, mark/3
FROM    Marks
```

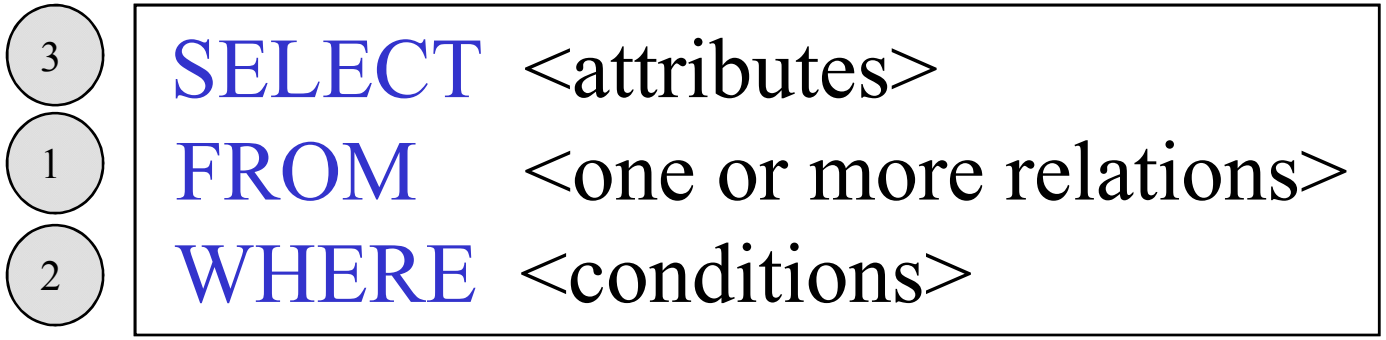
Null Values

- If x= NULL then x='Joe' is UNKNOWN

- In SQL there are three boolean values:

FALSE	=	0
UNKNOWN	=	0.5
TRUE	=	1

A tuple/row having only TRUE in WHERE clause is passed to SELECT



Null Values

- C1 AND C2 = min(C1, C2)
- C1 OR C2 = max(C1, C2)
- NOT C1 = 1 – C1

```
SELECT *
FROM Person
WHERE  (age < 25) AND
       (height > 6 OR weight > 190)
```

Tuple with the following values
age=20
height=NULL
weight=200
Will it be in the result?

Rule in SQL: include only tuples that yield TRUE

Answer: YES

Null Values

- C1 AND C2 = min(C1, C2)
- C1 OR C2 = max(C1, C2)
- NOT C1 = 1 – C1

```
SELECT *
FROM Person
WHERE  (age < 25) AND
       (height > 6 OR weight > 190)
```

Tuple with the following values
age=NULL
height=5
weight=200
Will it be in the result?

Answer: NO

Null Values

Unexpected behavior:

```
SELECT *
FROM    Person
WHERE  age < 25  OR  age >= 25
```

Some Persons are not included !

Null Values

Can test for NULL explicitly:

- x IS NULL
- x IS NOT NULL

```
SELECT *
FROM    Person
WHERE  age < 25  OR  age >= 25 OR age IS NULL
```

Now it includes all Persons

Aggregation

```
SELECT   AVG(price)
FROM     Product
WHERE    maker='Toyota'
```

```
SELECT  COUNT(*)
FROM    Product
WHERE   year > 1995
```

SQL supports several aggregation operations:

sum, count, min, max, avg

All aggregations apply to a single attribute (multiple attributes can be combined in a formula) [Except COUNT(*)]

NULL values are ignored [Except COUNT(*)]

COUNT(*) and COUNT(mark)

Any NULL in a formula will cause a NULL result, not a value of 0.

Marks

student	**mark**
Paul Smith	43
Rachel Sewell	57
Helen Treacy	32
Pinder Surree	NULL

```
SELECT COUNT(*)
FROM Marks
```

4

```
SELECT COUNT(mark)
FROM Marks
```

3

COUNT(*) and COUNT(mark)

Any NULL in a formula will cause a NULL result, not a value of 0.

Marks

student	mark
Paul Smith	43
Rachel Sewell	57
Helen Treacy	32
Pinder Surree	NULL
NULL	NULL

```
SELECT COUNT(*)
FROM Marks
```

5

```
SELECT COUNT(mark)
FROM Marks
```

3

Aggregation: Count

COUNT applies to duplicates, unless otherwise stated:

```
SELECT  COUNT(category)
FROM    Product
WHERE   year > 1995
```

same as COUNT(*)

We probably want:

```
SELECT  COUNT(DISTINCT category)
FROM    Product
WHERE   year > 1995
```

More Examples

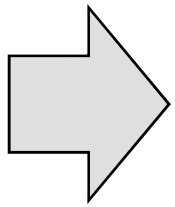
Find total sales

Purchase

Product	Day	Price	Quantity
Bagel	10/21/2005	1	20
Banana	10/03/2005	0.5	10
Banana	10/10/2005	1	10
Bagel	10/25/2005	1.50	20

Purchase

```
SELECT  SUM(price * quantity)
FROM    Purchase
```



65 (= 20+5+10+30)

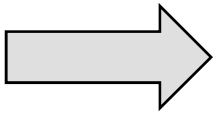
Simple Aggregations

Find total sales of product Bagel

Purchase

Product	Day	Price	Quantity
Bagel	10/21/2005	1	20
Banana	10/03/2005	0.5	10
Banana	10/10/2005	1	10
Bagel	10/25/2005	1.50	20

```
SELECT  SUM(price * quantity)
FROM    Purchase
WHERE   product = 'bagel'
```



50 (= 20+30)

Grouping and Aggregation

Find total sales after 10/04/2005 per product.

Product	Day	Price	Quantity
Bagel	10/21/2005	1	20
Banana	10/03/2005	0.5	10
Banana	10/10/2005	1	10
Bagel	10/25/2005	1.50	20

Purchase

Desired Outputs

Product	TotalSales
Bagel	50
Banana	10

Grouping and Aggregation

Find total sales after 10/04/2005 per product.

Product	Day	Price	Quantity
Bagel	10/21/2005	1	20
Banana	10/03/2005	0.5	10
Banana	10/10/2005	1	10
Bagel	10/25/2005	1.50	20

Purchase

```
SELECT    product, SUM(price*quantity) AS "Total Sales"
FROM      Purchase
WHERE     day > '04-OCT-05'
GROUP BY  product
```

Let's see what this means…

Grouping and Aggregation

Find total sales after 10/04/2005 per product.

```
SELECT    product, SUM(price*quantity) AS "Total Sales"
FROM      Purchase
WHERE     day > '04-OCT-05'
GROUP BY  product
```

1. Compute the FROM and WHERE clauses.

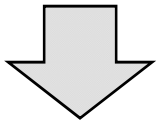
2. Group by the attribute(s) in the GROUP BY

3. Compute the SELECT clause on grouped attributes and aggregates.

1. FROM-WHERE

Find total sales after 10/04/2005 per product.

Product	Day	Price	Quantity
Bagel	10/21/2005	1	20
Banana	10/03/2005	0.5	10
Banana	10/10/2005	1	10
Bagel	10/25/2005	1.50	20



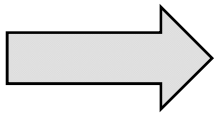
Product	Day	Price	Quantity
Bagel	10/21/2005	1	20
Banana	10/10/2005	1	10
Bagel	10/25/2005	1.50	20

2. GROUP BY

Product	Day	Price	Quantity
Bagel	10/21/2005	1	20
Bagel	10/25/2005	1.50	20
Banana	10/10/2005	1	10

3. SELECT

Product	Day	Price	Quantity
Bagel	10/21/2005	1	20
Bagel	10/25/2005	1.50	20
Banana	10/10/2005	1	10



Product	TotalSales
Bagel	50
Banana	10

```
SELECT    product, SUM(price*quantity) AS "Total Sales"
FROM      Purchase
WHERE     day > '04-OCT-05'
GROUP BY  product
```

HAVING Clause

Find all products and their total sales which were sold at least 30 pieces after 10/04/2005.

```
SELECT      product, SUM(price * quantity)
FROM        Purchase
WHERE       day > '04-OCT-05'
GROUP BY    product
HAVING      SUM(quantity) >= 30
```

HAVING clause contains conditions on aggregates.

General form of Grouping and Aggregation

SELECT S
FROM $R_1,\ldots,R_n$
WHERE C1
GROUP BY $a_1,\ldots,a_k$
HAVING C2



S = may contain attributes $a_1,\ldots,a_k$ and/or any aggregates but NO OTHER ATTRIBUTES

C1 = is any condition on the attributes in $R_1,\ldots,R_n$

C2 = is any condition on aggregate expressions

General form of Grouping and Aggregation

```
SELECT    S
FROM      R1,…,Rn
WHERE     C1
GROUP BY  a1,…,ak
HAVING    C2
```

Evaluation steps:

1. Evaluate FROM-WHERE, apply condition C1
2. Group by the attributes $a_1,\ldots,a_k$
3. Apply condition C2 to each group (may have aggregates)
4. Compute aggregates in S and return the result

Product (maker, model, type)
PC (model, speed, ram, hd, price)
Laptop (model, speed, ram, hd, screen, price)

Solutions on next slides

Find the manufacturers that make at least three different models of PCs

Find all manufacturers that produce all three types of products (pc, printer, laptop)

Find all manufacturers that produce only one product type and more than one model

Find the average speed of PCs

Find the average price of PCs made by manufacturer 'A'

Find for each speed of PC above 2.0, the average price

Find for each manufacturer who sell's PCs the maximum price of a PC

Product (maker, model, type)
PC (model, speed, ram, hd, price)
Laptop (model, speed, ram, hd, screen, price)

Find the manufacturers that make at least three different models of PCs

Product (maker, model, type)
PC (model, speed, ram, hd, price)
Laptop (model, speed, ram, hd, screen, price)

Find the manufacturers that make at least three different models of PCs

```
SELECT      maker
FROM        Product
WHERE       type = 'pc'
GROUP BY    maker
HAVING      COUNT(model) >= 3
```

Product (maker, model, type)
PC (model, speed, ram, hd, price)
Laptop (model, speed, ram, hd, screen, price)

Find the manufacturers that make at least three different models of PCs

```
SELECT DISTINCT p1.maker
FROM Product p1, Product p2, Product p3
WHERE p1.type <> p2.type
AND p1.type <> p3.type
AND p2.type <> p3.type
AND p1.maker = p2.maker
AND p1.maker = p3.maker;
```

Product (maker, model, type)
PC (model, speed, ram, hd, price)
Laptop (model, speed, ram, hd, screen, price)

Find all manufacturers that produce all three types of products (pc, printer, laptop)

```
SELECT      maker
FROM        Product
GROUP BY    maker
HAVING COUNT(DISTINCT type) = 3
```

Product (maker, model, type)
PC (model, speed, ram, hd, price)
Laptop (model, speed, ram, hd, screen, price)

Find all manufacturers that produce only one product type and more than one model

```
SELECT      maker
FROM        Product
GROUP BY    maker
HAVING COUNT(DISTINCT type) = 1
AND    COUNT(model) > 1
```

Product (maker, model, type)
PC (model, speed, ram, hd, price)
Laptop (model, speed, ram, hd, screen, price)

Find the average speed of PCs

```
SELECT    AVG(speed)
FROM      PC
```

Product (maker, model, type)
PC (model, speed, ram, hd, price)
Laptop (model, speed, ram, hd, screen, price)

Find the average price of PCs made by manufacturer ‘A’

```
SELECT    AVG(price)
FROM      Product, PC
WHERE     Product.model = PC.model
AND       maker  = ‘A’
```

Product (maker, model, type)
PC (model, speed, ram, hd, price)
Laptop (model, speed, ram, hd, screen, price)

Find for each speed of PC above 2.0, the average price

```
SELECT      speed, AVG(price)
FROM        PC
WHERE       speed > 2.0
GROUP BY    speed
```

Product (maker, model, type)
PC (model, speed, ram, hd, price)
Laptop (model, speed, ram, hd, screen, price)

Find for each manufacturer who sell's PCs the maximum price of a PC

```
SELECT      maker, MAX(price)
FROM        PC, Product
WHERE       PC.model = Product.model
GROUP BY    maker
```